

Data for Robot Learning

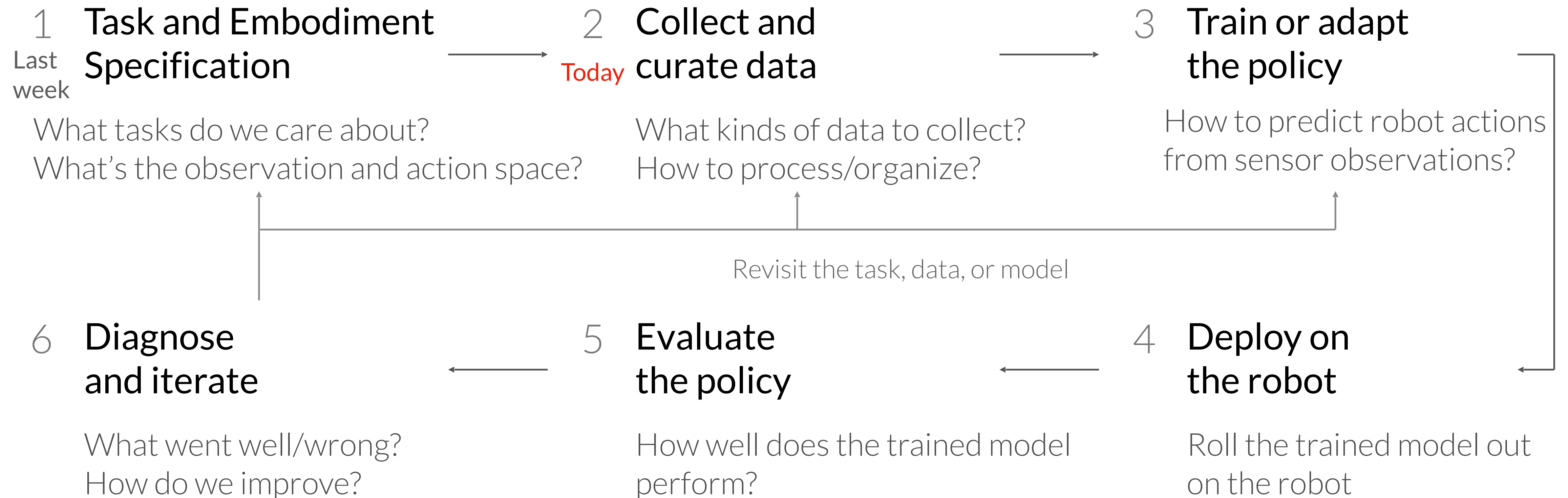
EN.601.[4|6]98 Hands-on Robot Learning

Krishna Murthy Jatavallabhula

Administrivia

- Schedule change: in-class robot assembly (assignment 0) is now Oct 1 (instead of Sep 24)
- Teams for course project formed — please look out for a team introduction email within the next day from the course staff
- CA office hour schedules:
 - Aleks: 6 pm on Tuesdays
 - Xinyi: 11 am on Wednesday
 - Please reach out to them (or me) about location
- Please meet-and-greet your team members before the next lecture

The *modern* robot learning lifecycle



Today's focus: Data for robot learning

- Three major sources of data (robots, humans, simulation)
- Popular paradigms for robot data collection (kinesthetic teaching, leader-follower teleoperation, wearable/handheld devices, hand/body tracking)
- Implications of the robot data collection method used on downstream learning
 - joint-level teleop vs end-effector control
 - data volume / scalability impacts
 - how the nature of downstream robot learning model impacts data strategy (e.g., VLA-vs-WAM)
- i.e., a lot to do about how data is collected for robot learning

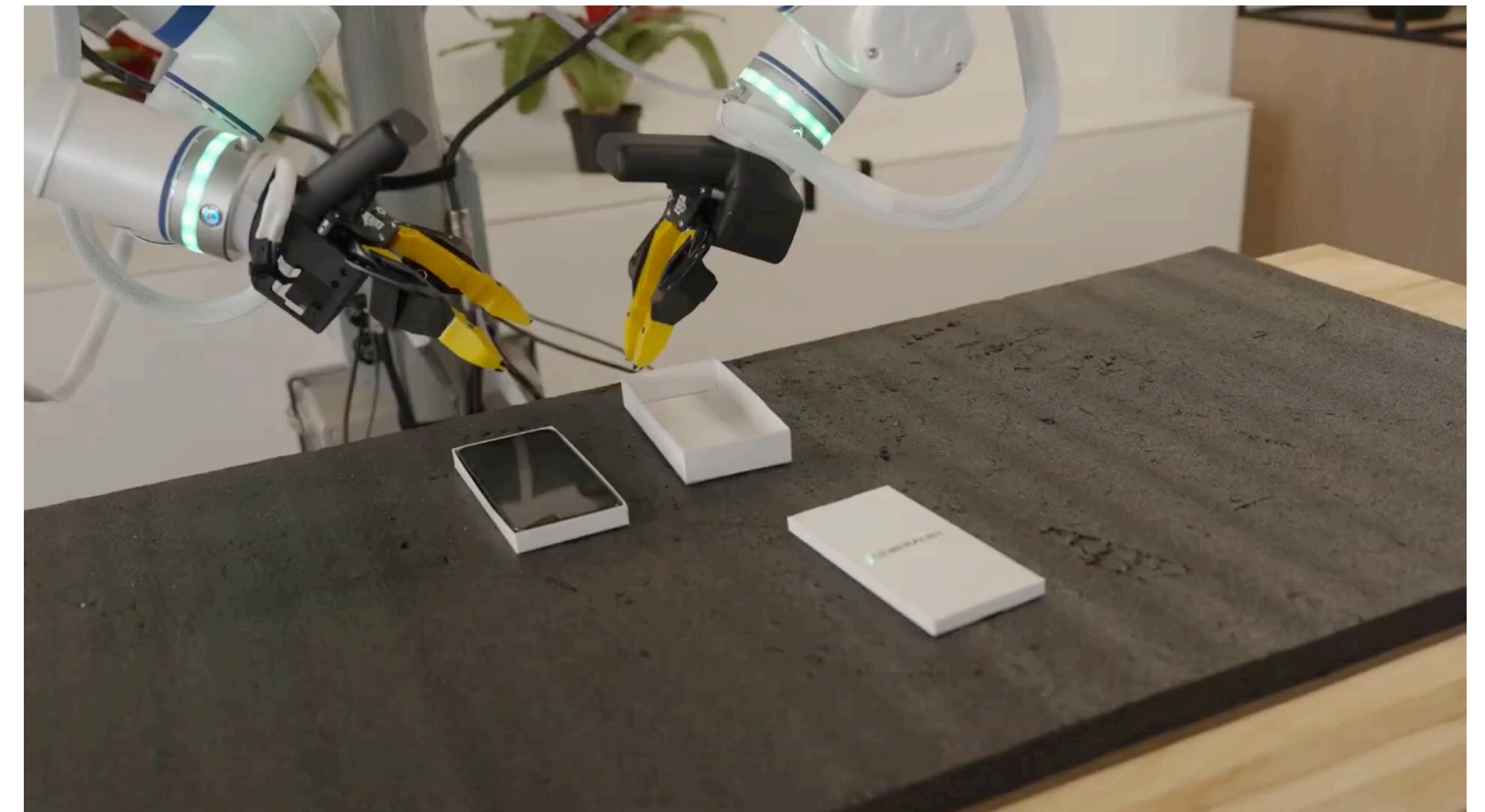
By the end of the lecture, you'll be able to

- Understand how joint-angle teleoperation works in a leader-follower setup
- Contrast various data collection methods and characterize their trade-offs given a downstream use-case for the collected data
- Understand the terms GELLO, UMI and what they mean (to the extent you'd be able to build / deploy similar devices)
- Analyze a modern robot learning dataset (e.g., a dataset stored in LeRobot format), browse the data schema, and relate each field to concepts we discuss today
- Assess how low-level implementation (design/controller) details impact data (and model) quality
- Build the conceptual tooling necessary to deep-dive into research papers describing data strategies

How we *teach* robots today



Mobile Aloha
Stanford



Generalist AI

How we *teach* robots today

- Collect a number of *demonstrations* of a robot (or a human, or hybrid) doing a task
- The number of demos (usually) ranges from $O(10)$ to $O(100)$ based on the nature of the task and the robot learning model being used
- Use these demonstrations to *train* a robot *policy* (a mapping from sensor observations to robot actions, e.g., joint angles)
- The (well, one) *hope/bet* is a scaling hypothesis
 - when trained on sufficiently large volumes of such data, newer tasks will require substantially fewer demos (e.g., $O(1)$, or none)
- Several bets in play on *how* to collect data for robot learning, or *what kind* of model to train

* non-exhaustive (e.g., simulation is another important source)

Main data paradigms for robot learning today*



Robot data

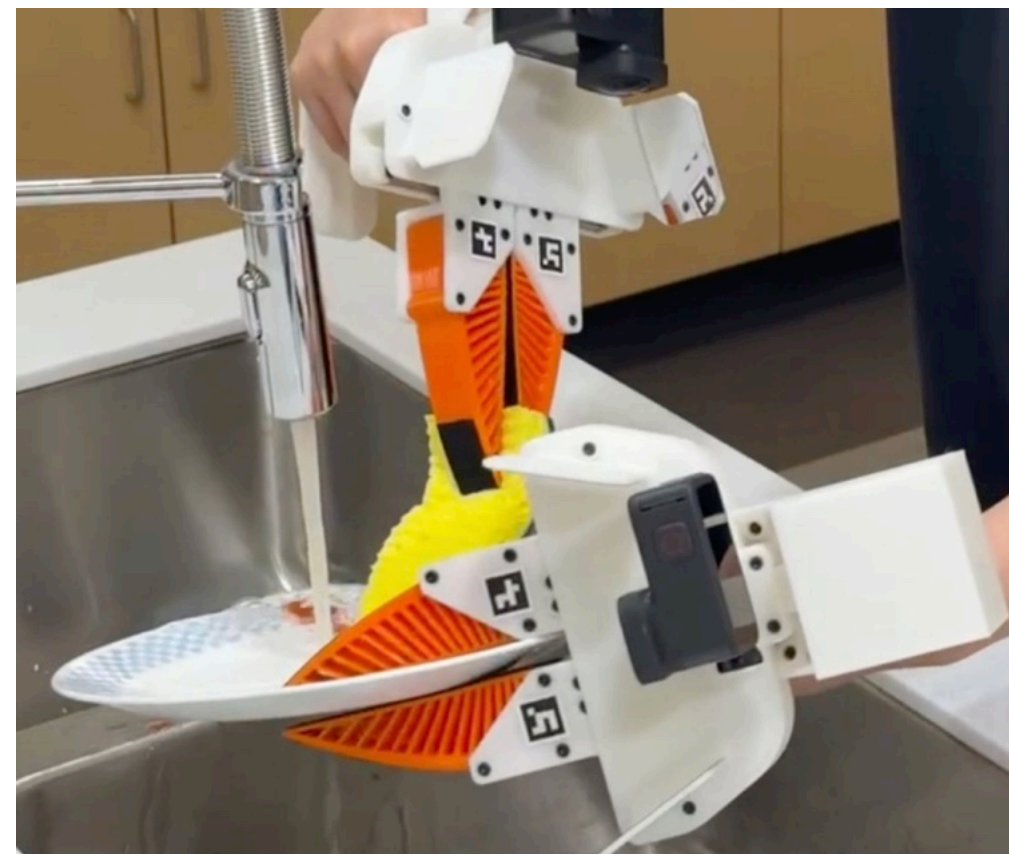
(Joint-level teleoperation)

Pros:

- + Kinematic feasibility
- + Fully-specified trajectories

Cons:

- Need kinematic twin of robot (at least a scaled-down one)
- Needs specialized infrastructure (harder to scale)



Hybrid data

(Wearable interfaces)

Pros:

- + Easy to collect in-the-wild
- + Better scalability

Cons:

- Data collection needs careful management by operator
- Restrictions on robot end-effectors



Human data

(Hand / body-tracking)

Pros:

- + Easy to collect in-the-wild
- + Highly scalable

Cons:

- Data needs to be retargeted to robot body
- Data quality is noisier than other paradigms

Robot data (joint-level teleoperation)

What we mean by 'robot data' in this class

- Data you'd obtain when there's actually a robot in-the-loop doing the task
- That is, for every input observation, there is a set of corresponding actions that we know are kinematically feasible on the robot
 - by construction, because the robot was involved when said data was collected
- The underlying mechanics by which the robot is controlled can vary (a lot). E.g.,
 - Leader-follower teleop: A kinematic twin of the robot is controlled directly by the operator, and the resulting configuration is relayed to the robot
 - End-effector controller teleop: The operator controls (moves) a device (such as a VR headset controller or a motion tracker), which then commands an end-effector position controller to move the robot
 - Kinesthetic teaching: Robot is moved through a (sparse) sequence of joint positions. A motion planner then interpolates these positions and the resultant trajectory is replayed on the robot
- In this class, we will (primarily) use leader-follower teleop with a $\sim 1:1$ kinematic twin

Taxonomy of *robot data* collection approaches

Method	What a teleoperator (human) does	Assumptions / Limitations
Kinesthetic teaching	Physically moves the robot into multiple configurations	Robot motors need to be backdrivable
Teach pendant / waypoint controllers	Selects poses (and other constraints like velocities) on an interface	Sparse waypoints need interpolation, feasibility, and safety checks
Handcrafted controller / planner	Writes code for a controller that takes in robot state + observations and does a task	Need to do anew for each task; Often assumes privileged info; Adequate state coverage is hard to handcraft
Leader–follower teleoperation	Moves a <i>leader</i> robot (the <i>follower</i> robot tracks the leader robot position/velocity)	Need a leader robot that is kinematically equivalent to the robot we want to teleop
Task-space teleoperation	Provides EEF motion, gripper commands	IK needs to work reliably across the workspace, resolve redundancy/singularities

Structure of a demonstration / episode

- Observations: images, proprioception (joint angles), and optionally joint velocities, gripper states, and other onboard sensors (force/torque, tactile, etc.)
- Actions: *commanded* joint position targets (and all conversion metadata — coordinate frames, timestamps, interpolation modes, etc.)
- Metadata / task context (as applicable): language instructions, embodiment details, low-level controller details, calibration data, software/firmware info
 - Not all people/organizations log all this, but we ought to
- Quality metrics: operator's assessment of trajectory quality, any notable 'events' such as retries or recovery, reasons for aborting
- Note: actions log *commanded* joint position targets, while observations log *actually realized* joint positions. Both are different, and not to be confused.

Commanded vs actual targets: Why? When to use?

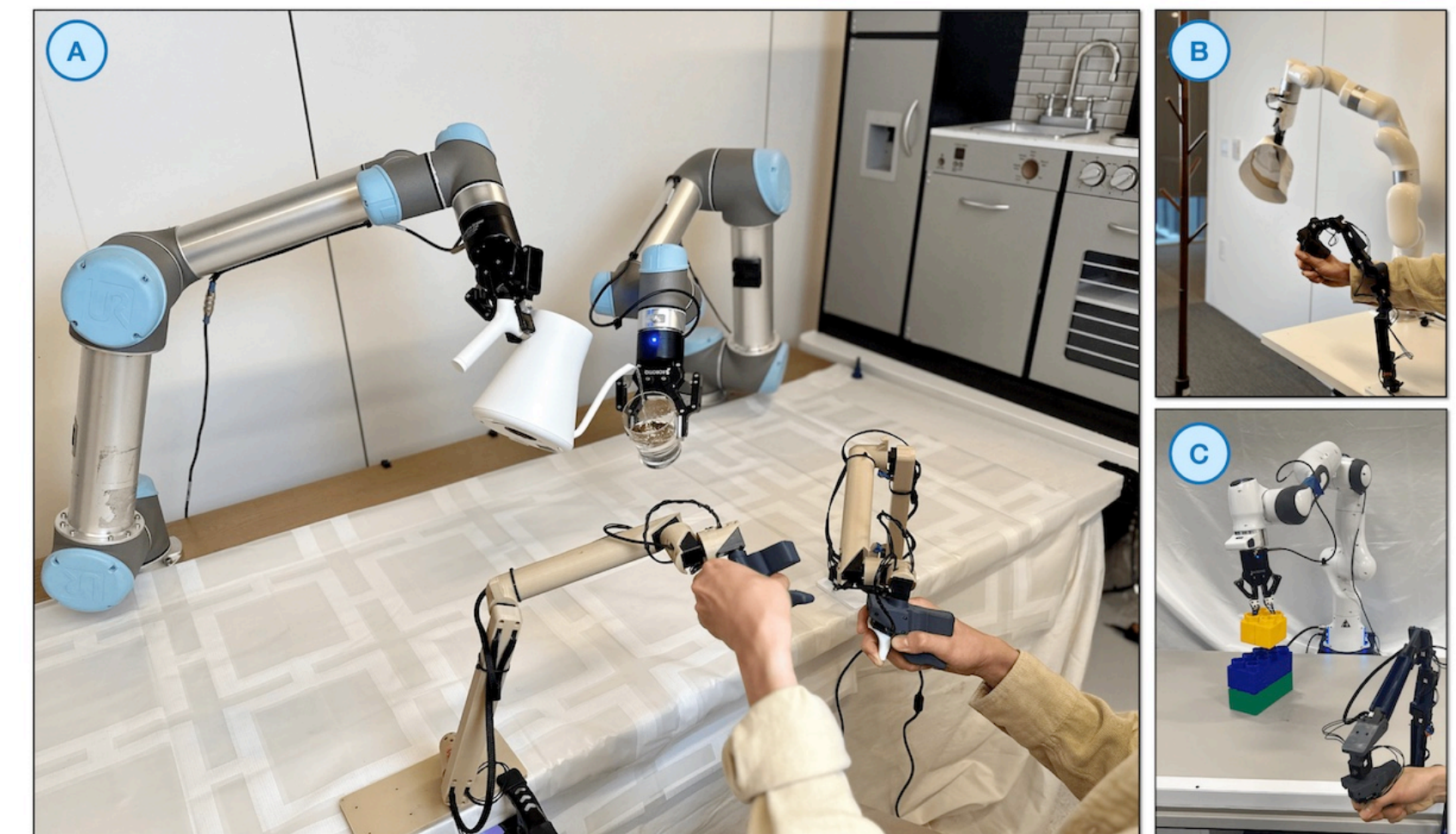
- Commanding a position target to a low-level controller does not mean the robot actually gets there
 - Actuation noise and low-level controller fluctuations always exist
 - In interactive teleoperation, the rate at which new joints stream to a robot are often faster than the rate at which the robot is able to realize them
- What happens when you train a policy on realized (as opposed to commanded poses)?
 - Policy learns to mimic *controller lag*
- When do we use realized poses (as opposed to commanded poses)?
 - As observations (proprioception)
 - (Later in the course) training dynamics / *world* models
- PSA: A lot of existing robot learning datasets declare an *action* field without ever specifying whether they're commanded or realized joint positions. The next time you explore a dataset, pay extra attention to this (seemingly irrelevant) detail — spare yourself of hours/days of agony.

Leader-follower teleoperation

- General idea: Human operator moves a *leader* arm, and the actual robot (a.k.a. the *follower* arm tracks the joint positions of the leader arm)
- In this class, we will use the SO-101 leader / follower arms as a primary mode of teleop data collection
- Leader arms typically replace the end effector with a gripper trigger
- The SO-101 leader has different motors compared to the leader. Why?
 - Cost, Backdrivability (ease of teleop)
- Leader arms needn't be the same form factor as the follower arm. GELLO uses a scaled-down form factor (these are compact, and preserve kinematic feasibility)
- One major drawback with popular leader-follower systems today is they're optimized for two-fingered grippers (extensions to dexterous hands need careful thought and redesign)



Homogeneous leader-followers



GELLO (xdof.ai)

Leader-follower teleop: Calibration

- We'll use this term often this semester
- Calibration refers to mapping the servo encoder positions to joint angle ranges.
- Recall: the STS3215 servos used in the SO-101 leaders and followers have a 12-bit encoder that provides the motor's position
- This position (0-4095) needs to be mapped to an angle (e.g., 0 - 180 degrees)
- Calibration needs to be performed whenever you unmount and remount a servo or its horn
- What happens if you don't do this?



[1] Motion Range vs Stats (threshold $\pm 2.0\sigma$)

Joint	Range	Mean	Std	Deviation	Offset	Status
shoulder_pan	2718	2725	32	-0.23 σ	-174	✓ PASS
shoulder_lift	2353	2350	77	+0.04 σ	710	✓ PASS
elbow_flex	2230	2222	9	+0.90 σ	-1659	✓ PASS
wrist_flex	2331	2329	17	+0.11 σ	-330	✓ PASS
wrist_roll	3857	4026	114	-1.48 σ	-555	✓ PASS
gripper	1483	1475	33	+0.23 σ	-845	✓ PASS

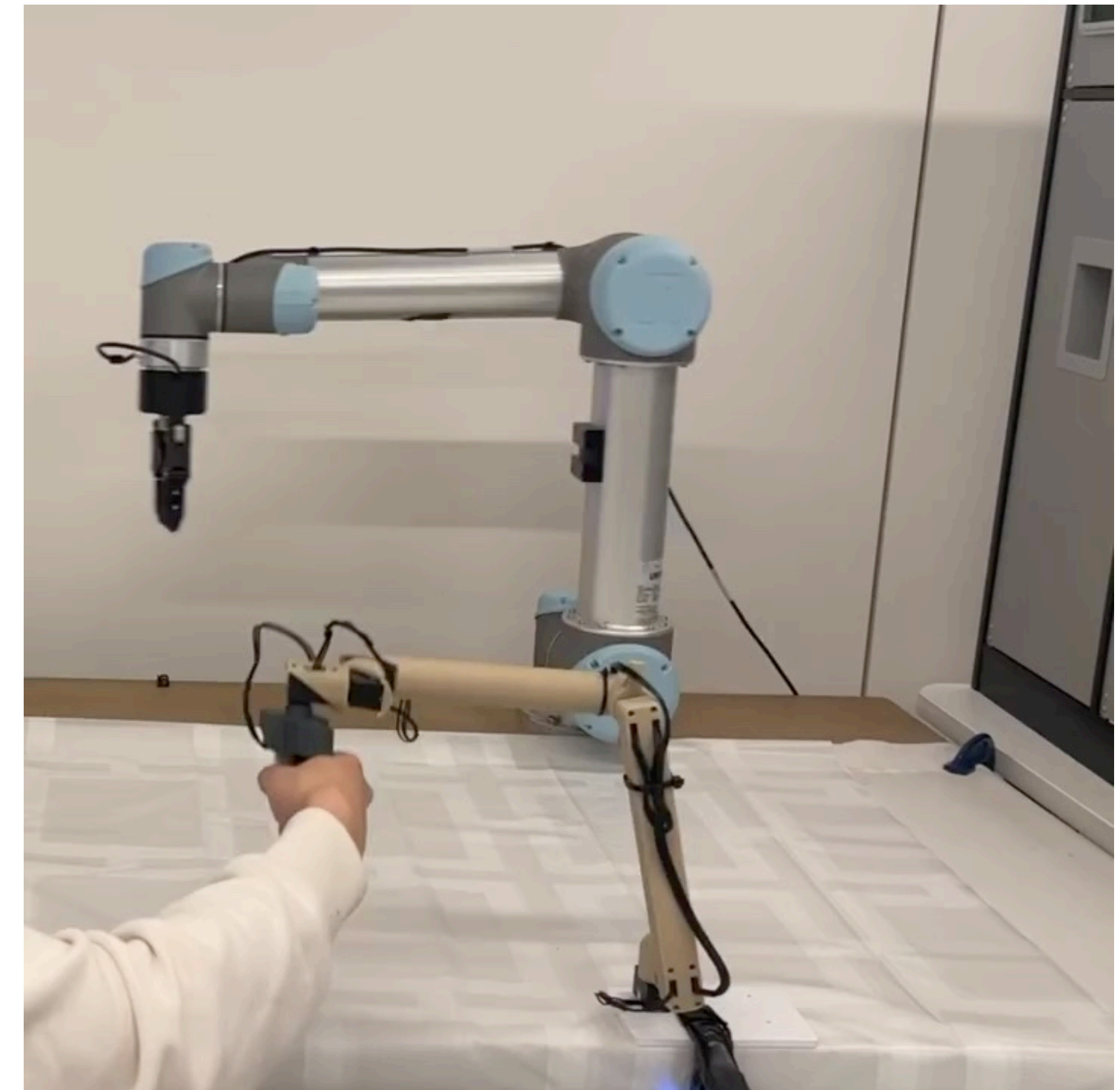
[2] Live Encoder Positions

Joint	Position	Calibrated Range	In Range
shoulder_pan	2174	857 - 3575	✓ OK
shoulder_lift	888	872 - 3225	✓ OK
elbow_flex	3059	861 - 3091	✓ OK
wrist_flex	1871	838 - 3169	✓ OK
wrist_roll	100	77 - 3934	✓ OK
gripper	1763	1727 - 3210	✓ OK

Overall: ✓ PASS - calibration looks good.

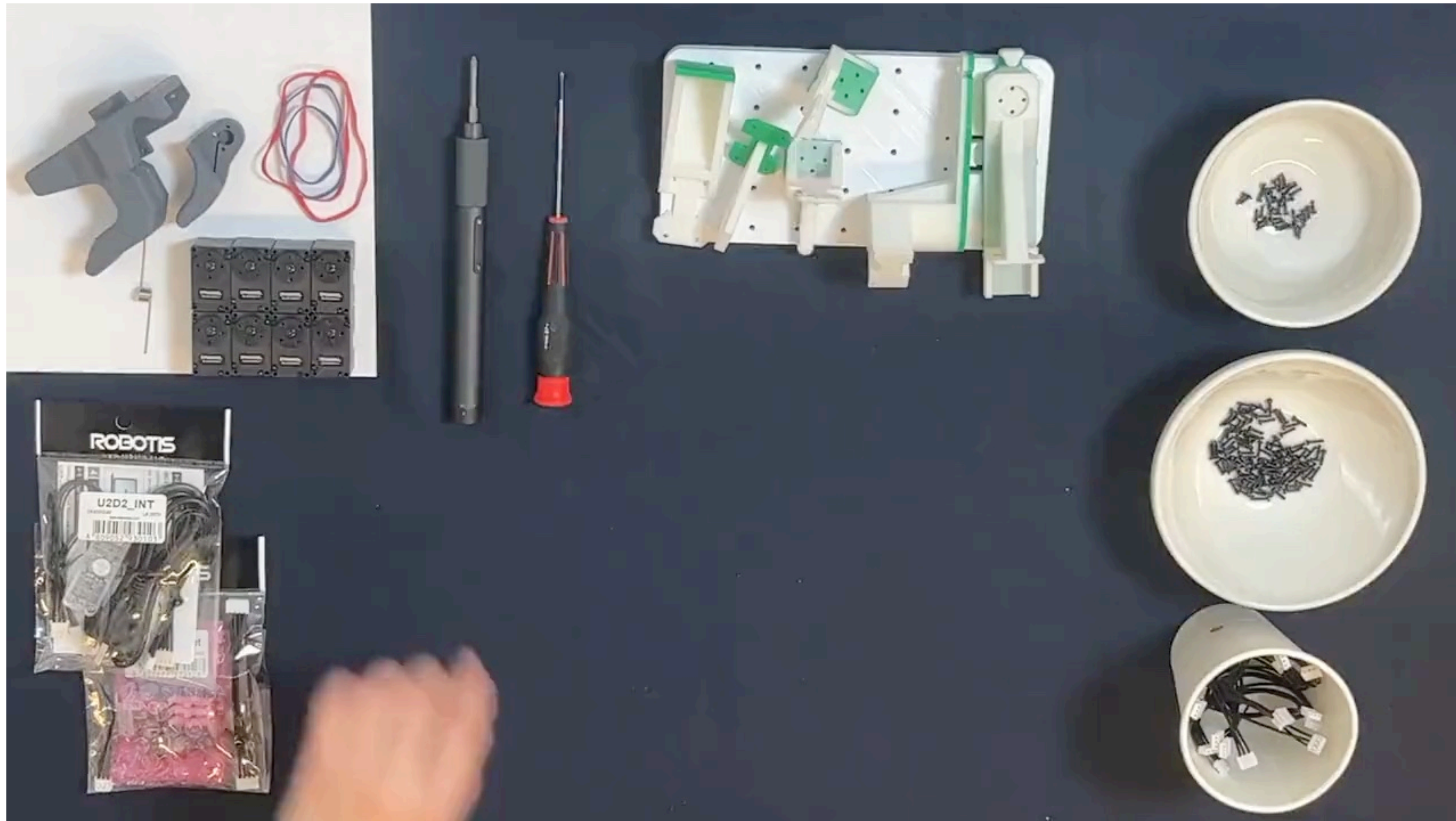
GELLO: low-cost and compact leader arms

- Most times, it is infeasible to get two copies of the same robot in a leader-follower setup (space, cost, back drivability)
- GELLO proposes building a compact leader arm with the same kinematic topology as the follower arm
- Off-the-shelf servo motors can be used as joint-angle sensors; and for gripper trigger
- Links can be 3D printed, and scaled down by a chosen scale factor, while preserving joint angles
- GELLOs can be *passive* or *active*:
 - Passive: servo motors do not produce any actuation — they're only used because we're interested in their position encoders
 - Active: servo motors compensate for gravity (i.e., if you let go of the leader arm, it stays in that pose w/o collapsing)
- Careful design choices: links need to ensure joint angle limits are near-identical to the robot (else, dead regions / joint limits)



Building a GELLO

By the end of this course, you'll be able to do it (it's similar to an SO-101 buildout)



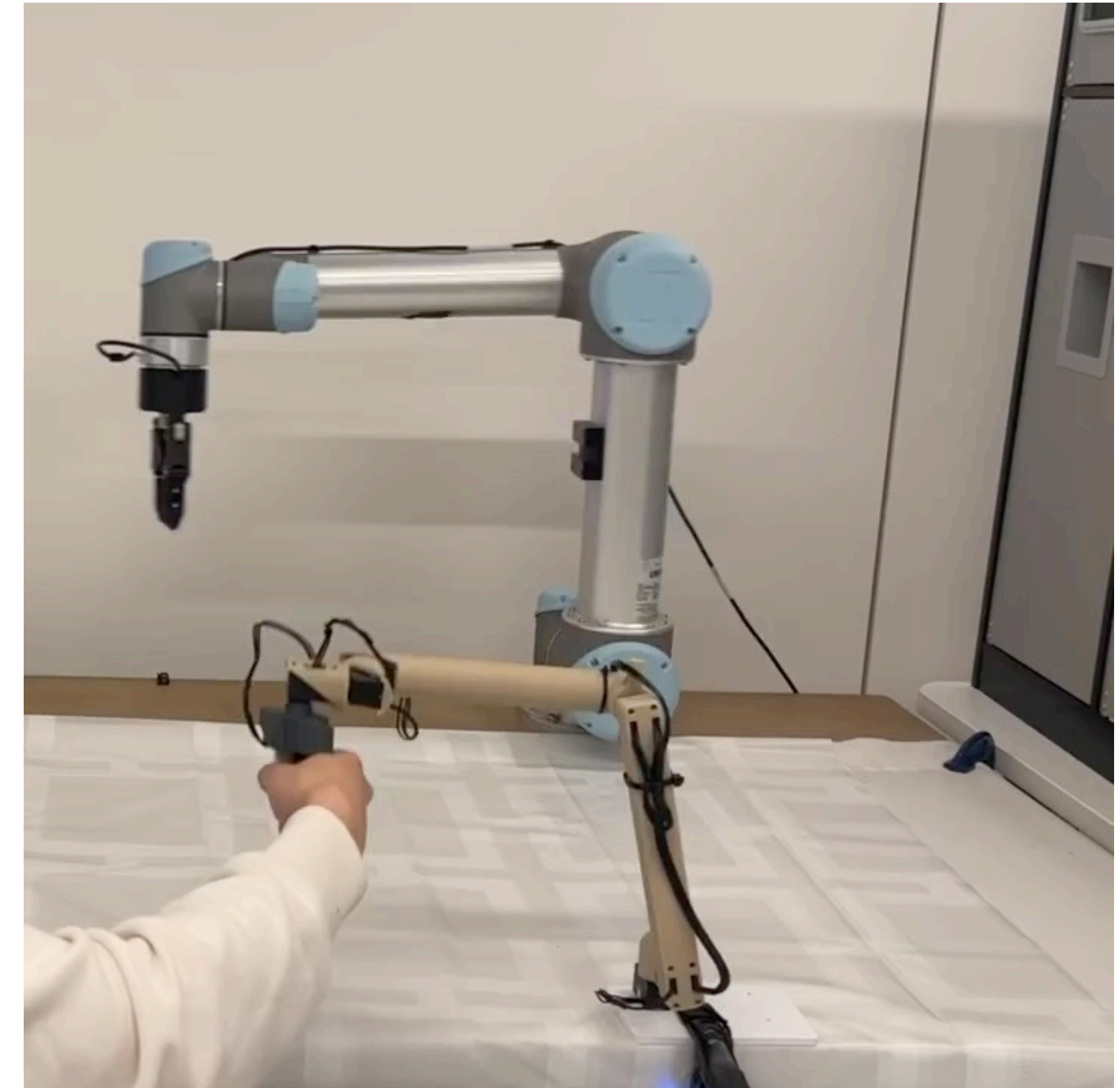
Mapping leader joint positions to a follower

$$q_{desired} = sq_{leader} + b$$

- Given leader joint position q_{leader} , how do we compute the follower position $q_{desired}$?
- Robot and GELLO encoders may have a different sign convention (robot convention is decided by robot manufacturer; GELLO convention is determined by the encoder in the motor you use).
- S (sign) denotes whether leader and follower joints measure the same sign / direction
- The offset (b) computes systematic offsets between leader and follower reference angles (i.e., account for robot and GELLO motors marking a different position on the rotor as origin/zero)

Practical considerations to factor in for a GELLO implementation

- 3D printed parts often snap — it's best to reinforce at least reinforce the most substantial load-bearing components (usually the part mounted to joint 2 in most GELLOs) by machining them out of metal
- Cable routing often imposes virtual joint limits / backlash due to tension
- Calibration is needed whenever a link / servo horn is unmounted / remounted
- How to initialize the GELLO for teleop? At startup, the leader and follower arms aren't usually at the same joint pose.
 - Approach 1: ensure both arms collapse to a 'home pose' before beginning teleop commands
 - Approach 2: run a position controller to gradually diffuse one of the arms to match the other
- Else, large / inconsistent motions may occur (unsafe, undesirable)



Gravity compensation

- Used when we want the arm to be teleoperated w/o always needing to support the arm's weight
- Delicate balance: we'd like the arm to stay put in-place, but also not feel too heavy to be moved around (we're teleoperating it after all)
- Often needs careful tuning (and system identification to estimate inertia and dynamics parameters)
- Gravity compensated leader arms feel different to teleop — they also produce different *kinds* of data (as compared to passive leader arms)
- Different data — in what way?

Gravity compensation: implementation considerations

- Used when we want the arm to be teleoperated w/o always needing to support the arm's weight
- Delicate balance: we'd like the arm to stay put in-place, but also not feel too heavy to be moved around (we're teleoperating it after all)
- Often needs careful tuning (and system identification to estimate inertia and dynamics parameters)
- Gravity compensated leader arms feel different to teleop — they also produce different *kinds* of data (as compared to passive leader arms)
- Different data — in what way?
- Multiple ways to implement gravity comp:
 - Passive spring/elastic mechanisms
 - Adding a counterweight
 - Active torque comp (computer motor efforts that need to be applied using a motor dynamics model)

Gravity compensation for a single link

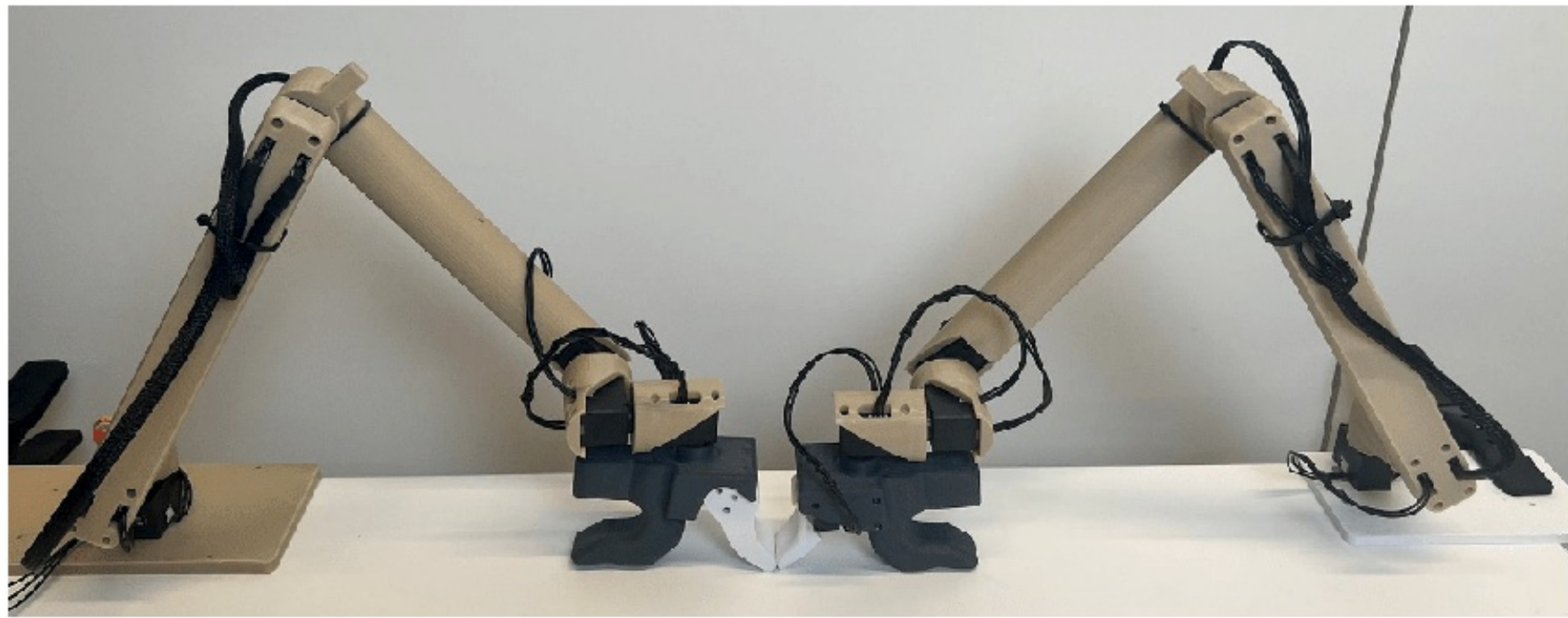
- Assume a link with 500 gm mass with center-of-mass 20 cm away from the joint q
- Downward force on the link is $F_{weight} = mg_0 = 0.5 \times 9.81 = 4.905N$
- Torque due to gravity is $\tau_{comp} = mg_0 r \cos(q)$ (since gravity is vertical, lever arm is $r \cos(q)$)
- So, required torque to support the link is based on joint angle
- Each link needs a different torque (e.g., links that shoulder more weight need higher torque), and this depends on the robot pose
- But all this is presuming the arm is static. If the arm is moving, we also need to counter velocity and other dynamic effects.
- More generally, controllers use $\tau_{motor} = \hat{g}(q) - B\dot{q}$
 - $\hat{g}(q)$ is torque needed to balance gravity
 - $B\dot{q}$ adds some damping torque (resists velocity; helps 'settle' the arm); B controls how aggressive the resistance is
 - Optionally (if we require *bilateral* teleop), this can be incorporated into a position hold controller to hold specific pose

Gravity compensation (contd...)

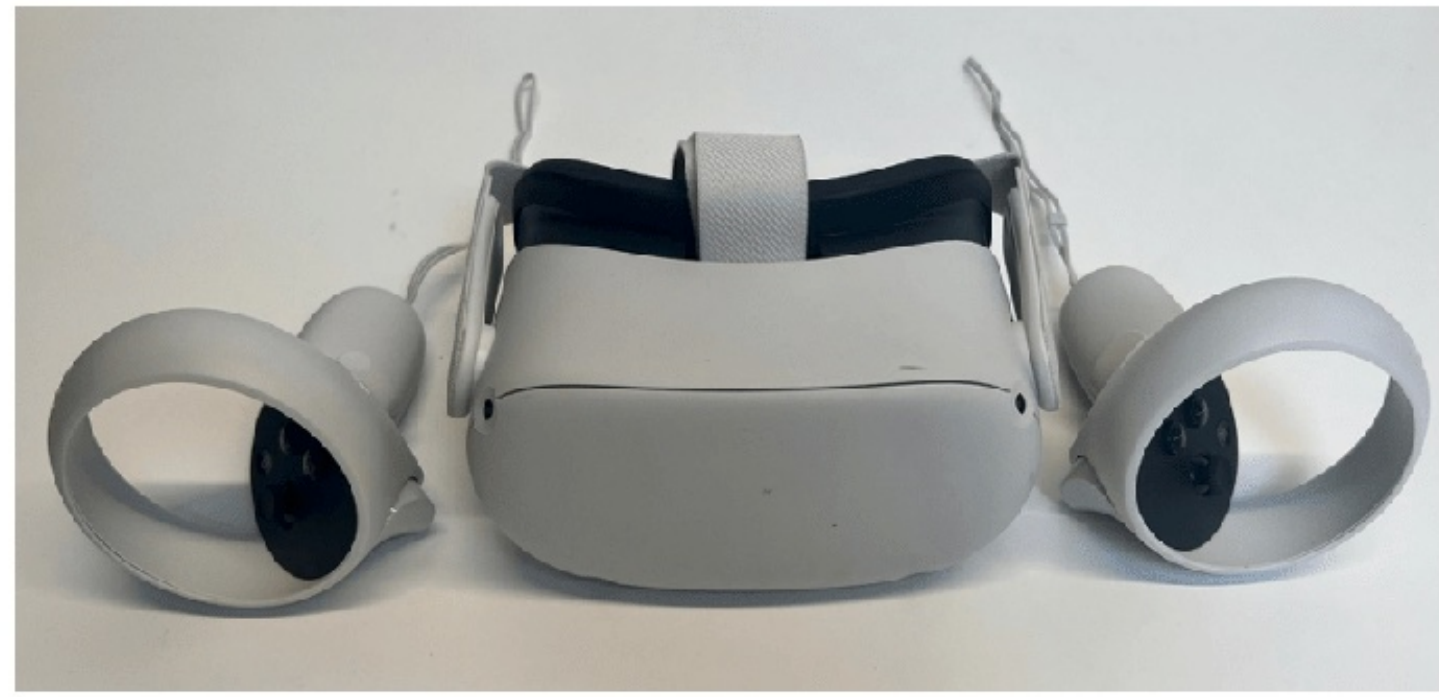
- Once torques have been calculated, we need to estimate the current needed to deliver that torque to the motor
- Recall the motor characteristic curve from last class (current draw depends on temperature, pose, payload, friction, and more)
- The SO-101 arms don't support this directly because the FeeTech STS3215 motors only support position control
- Other kinds of smart servos (e.g., some of the Dynamixel servos) support current control, and are preferred for active, gravity-compensated GELLO setups
- How do we then mimic gravity-comp-like behavior on the SO-101 leader arms?
 - Position hold (i.e., track leaders last position and continue holding it using a position controller)
 - Low torque and gear ratio leader arms (easier to move about)

Other common *robot data* teleop devices

- Which of these interfaces fully (unambiguously) specifies robot pose?
- Which of these devices is the easiest to teleoperate?
- Which of these devices is the cheapest?



GELLO (xdof.ai)



VR headset/controllers



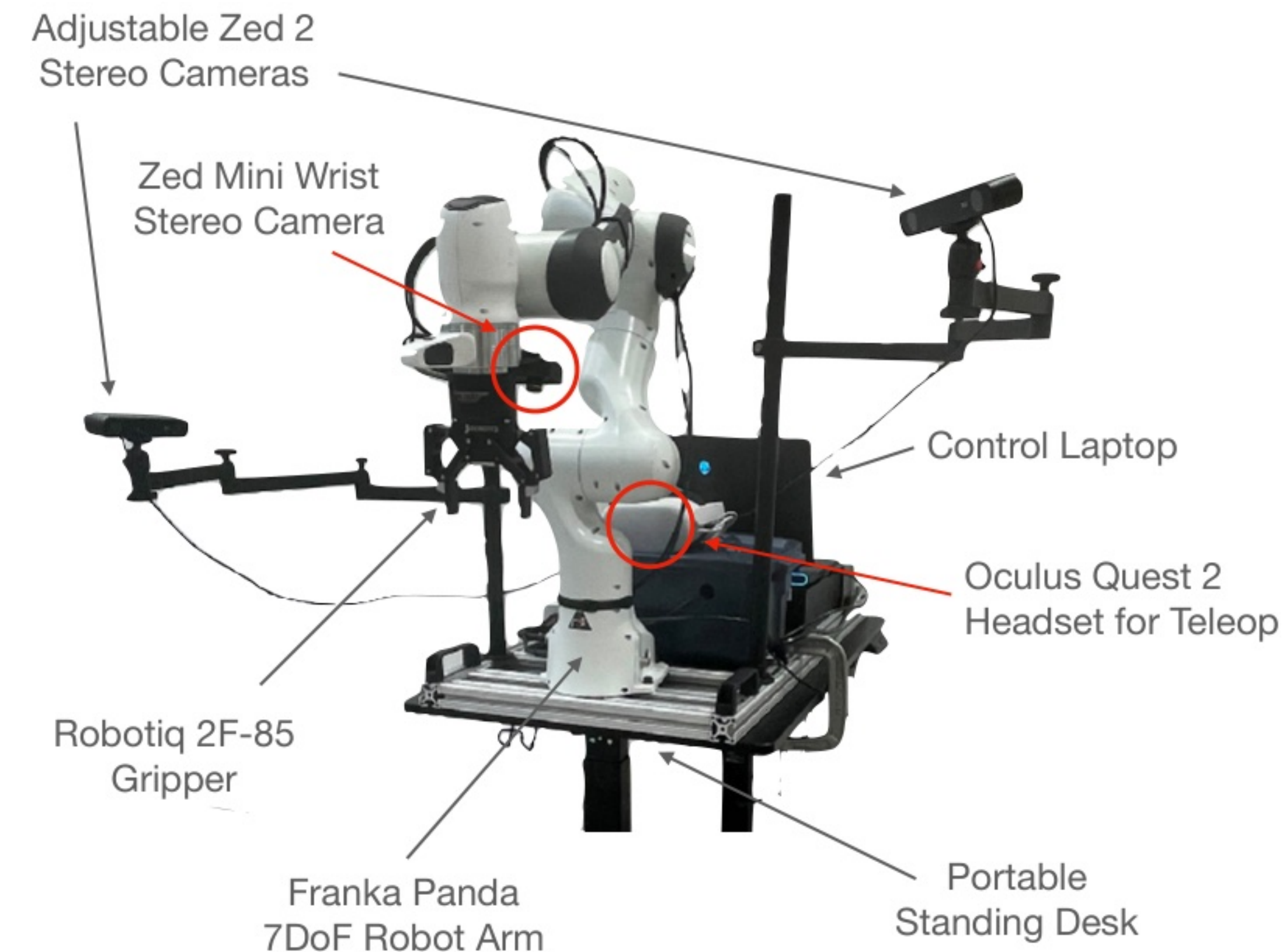
Spacemice

End-effector position control for teleoperation

- Interfaces like mixed-reality controllers/headsets are intuitive / easy to use
- However, they don't fully (and unambiguously) specify robot poses
- Instead, they provide a change in operator controller position over time
- These need to then be mapped into a robot frame, scaled down to safe displacements and speeds, checked for position/velocity/torque limits, possibly interpolated for smoothness, and then relayed to the robot
- Redundant manipulators have more than one way of reaching target joint positions
- IK solvers can run into singularities
- *All* these low-level choices (interpolation, IK, singularity handling, limits) will inevitably bias data quality and impact trained models downstream
- One significant advantage of end-effector position control is that the generated command can be used in robot-agnostic (or cross-embodiment) pertaining data mixtures w/o substantial post-processing

Case-study: Smart design choices implemented by the DROID teleop controller*

- DROID setup includes a table-top mounted Franka robot arm (7 DoF)
- Teleop was performed using a Oculus Quest 2 controller
- Design choice 1: make the robot respond slower to the controller (counterintuitive, but works well when data collection is distributed across multiple geographic locations). Operators get 'used to' robot slowness, leading to consistent speeds
- Design choice 2: scale all commands such that the joint pose differences across consecutive frames always have a consistent magnitude. This results in a well-behaved action distribution
- As a result, we observe that policies trained on DROID that use the raw joint poses perform better compared to ones that post-process DROID to produce end-effector poses



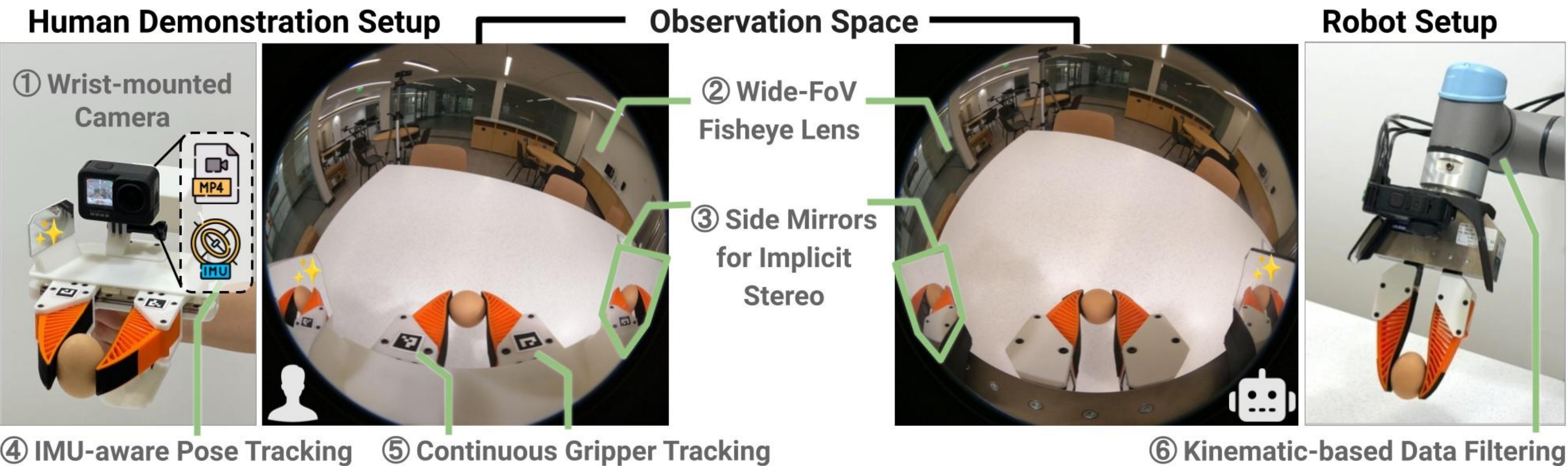
*KMJ: I'm unsure if these design choices were intentional, but they sure were smart. (I wasn't involved in the DROID dataset effort)

Hybrid data (wearable end-effectors)

Hybrid data (wearable end-effectors)

- Leader-follower setups aren't always practicable (expensive, bulky, what if we'd like to collect data in the wild?)
- Another dominant paradigm is to design wearable end-effectors that human teleoperators can *wear* on their hand/arm/torso/etc. and collect data
- The same end effector can then be mounted on a robot, enabling transfer
- Popular paradigm, being scaled by Generalist AI, Sunday Robotics, and others.

Universal Manipulation Interface (UMI)

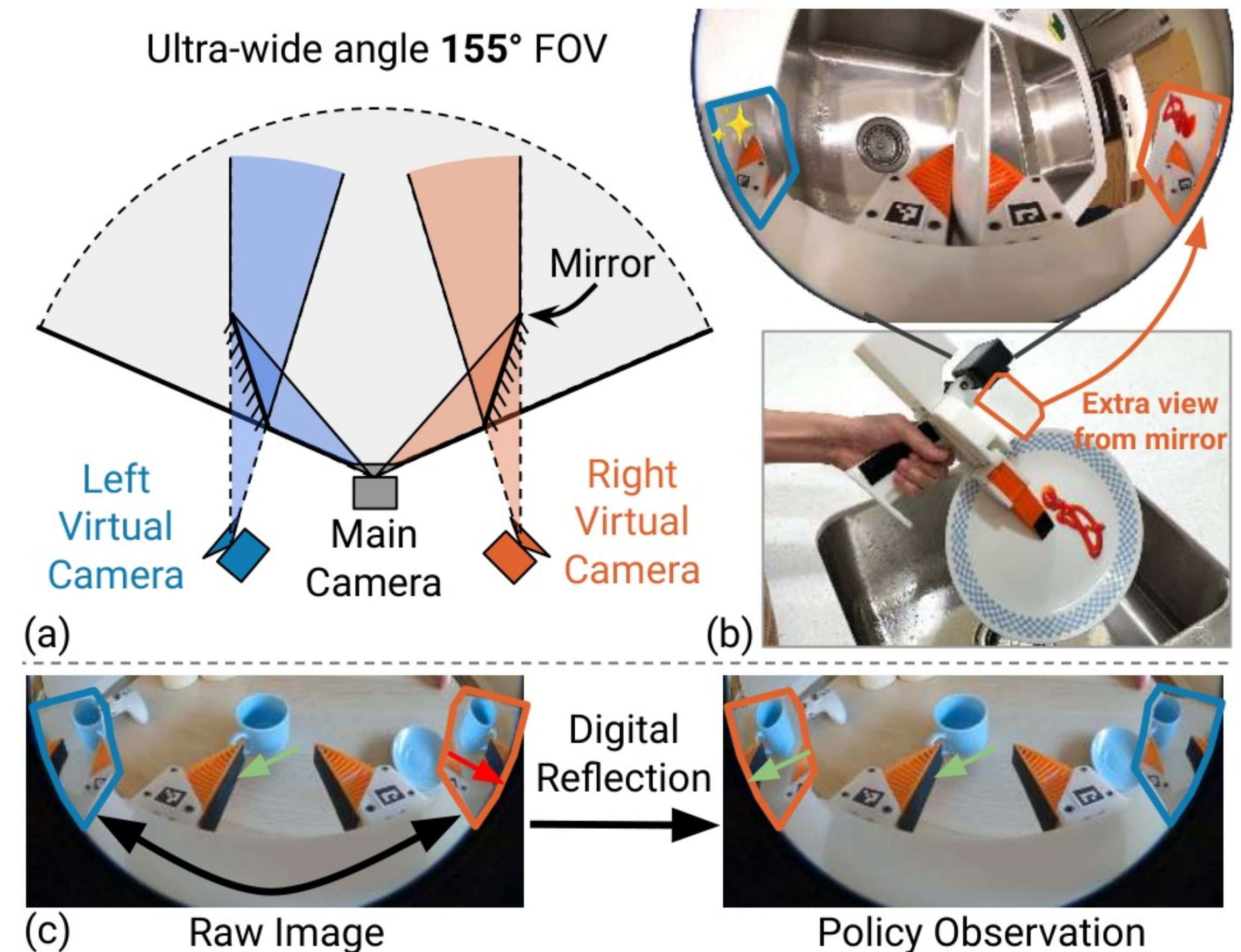


UMI: motivation + design

- Collect real object interactions with a handheld parallel-jaw gripper
- Use a wrist-mounted camera view during collection and robot deployment
- Reconstruct tool motion and gripper actions, then train a visual policy to predict future trajectories
- UMI enables contact sensing inherently via the wearable
- Careful co-design needed (hardware, odometry/SLAM, action representations, care during deployment)
- Design:
 - Use a go-pro camera to capture video and inertial measurements (IMU sync. Issues solved)
 - Camera-to-tool extrinsic calibration transforms camera trajectories to tool trajectories

UMI mirror geometry, virtual cameras

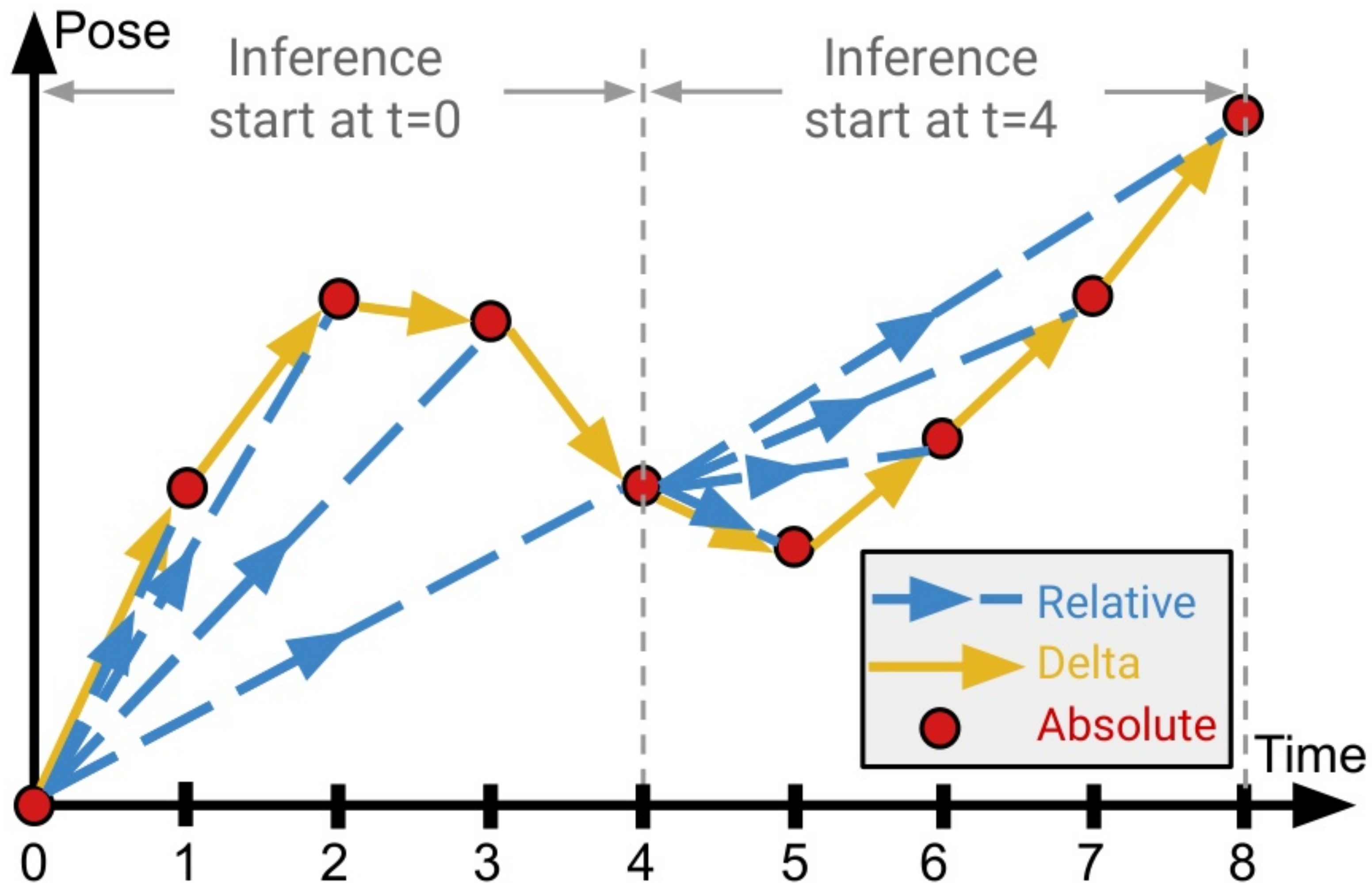
- Reduce occlusions, without adding more cameras
- Use mirrors on the sidewalls!
- Works as *implicit* stereo
- Improves downstream policy performance



UMI: practical considerations

- Gripper design impacts the kind of grasp actions feasible
- Human operator senses contacts; but these aren't measured/logged
- Human arm/wrist joint structure and constraints are very different from a robots (e.g., a human may be able to reach tight spaces that the robot wouldn't)
- Humans operators also have inherent speed differences when performing a task
- Camera mounts moving a couple mm is enough to break a policy trained on the data
- Images may be blurry. Visual tracking / SLAM may break
- Error / drift accumulates over time
- Companies that work on UMI data need to account for all these variables

UMI: relative vs delta trajectories



Bimanual UMI?

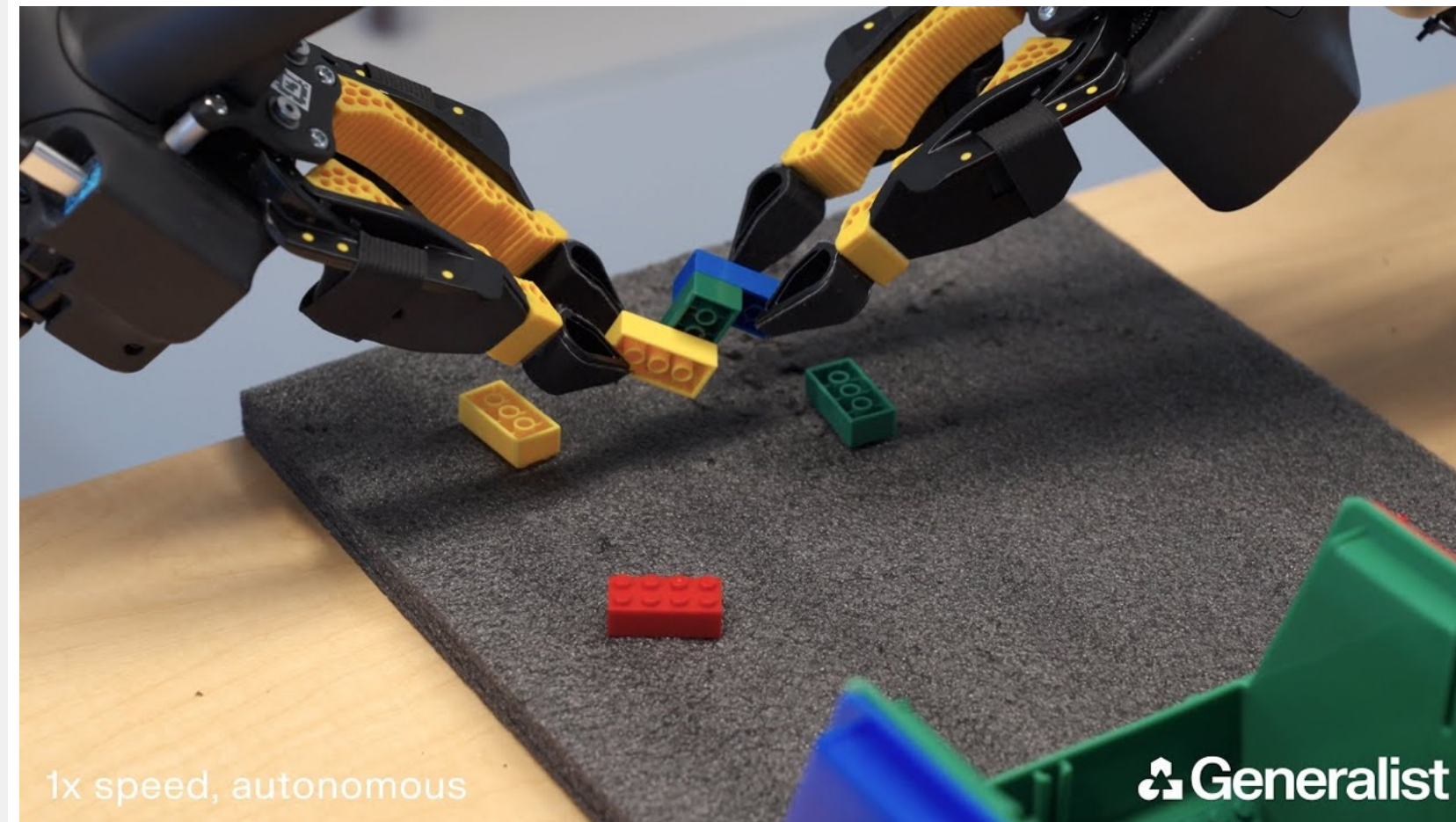
- Collect demos with one gripper in each hand
- Both trajectories need to be expressed relative to a unified coordinate frame
- The trajectories also need to be time-synchronized
- Approach: collect both demonstration videos, localize them against the same scene map
- SLAM accuracy becomes even more crucial



Several variants of wearables, beyond UMI



Sunday Robotics

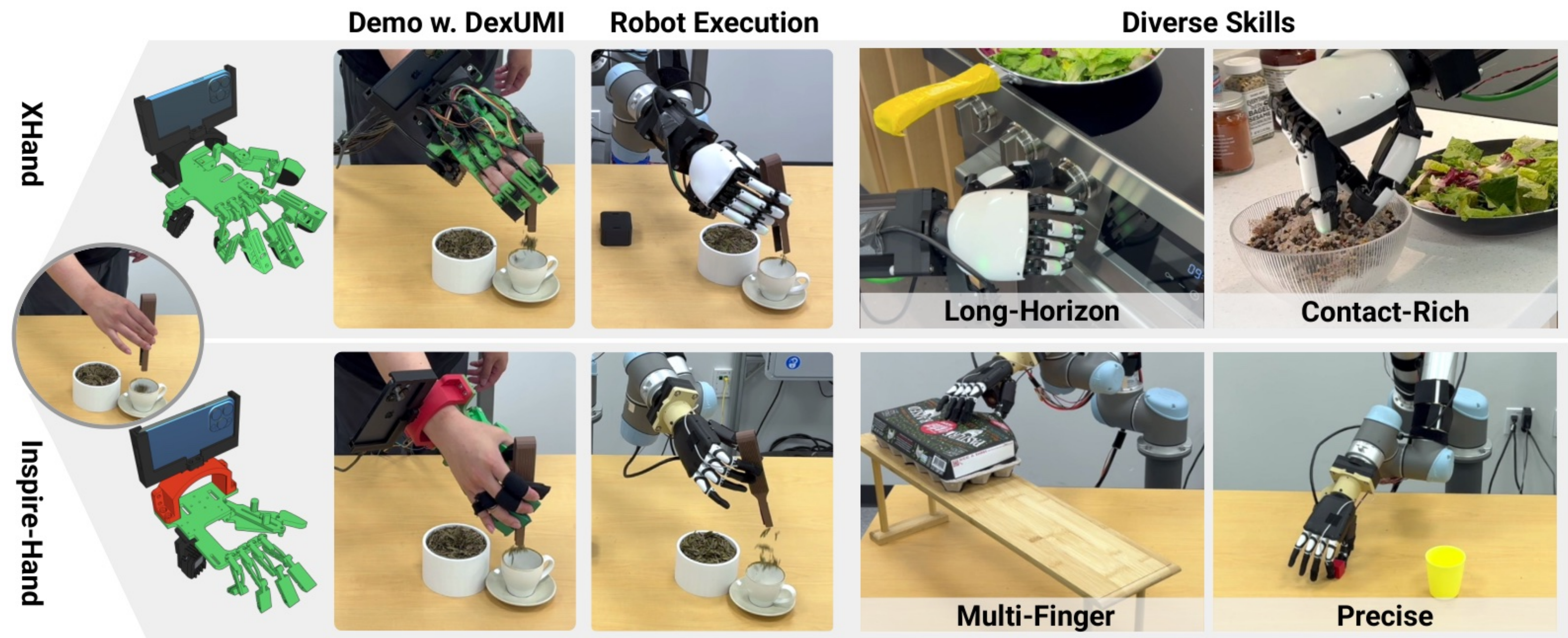


Generalist AI



Reward AI

Dexterous hands with UMI



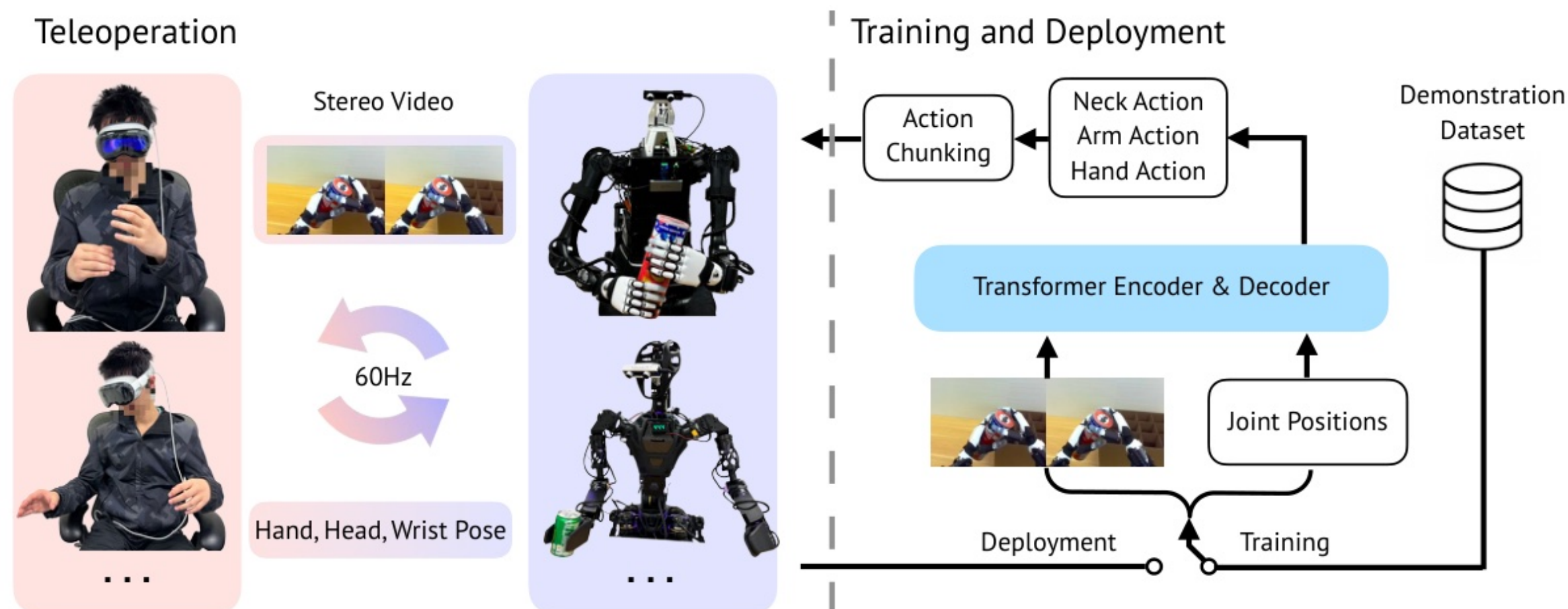
Hardware and transfer examples from DexUMI.

Human data for robot learning

Particularly relevant for: Dexterity, Scalability, Ease-of-collecting

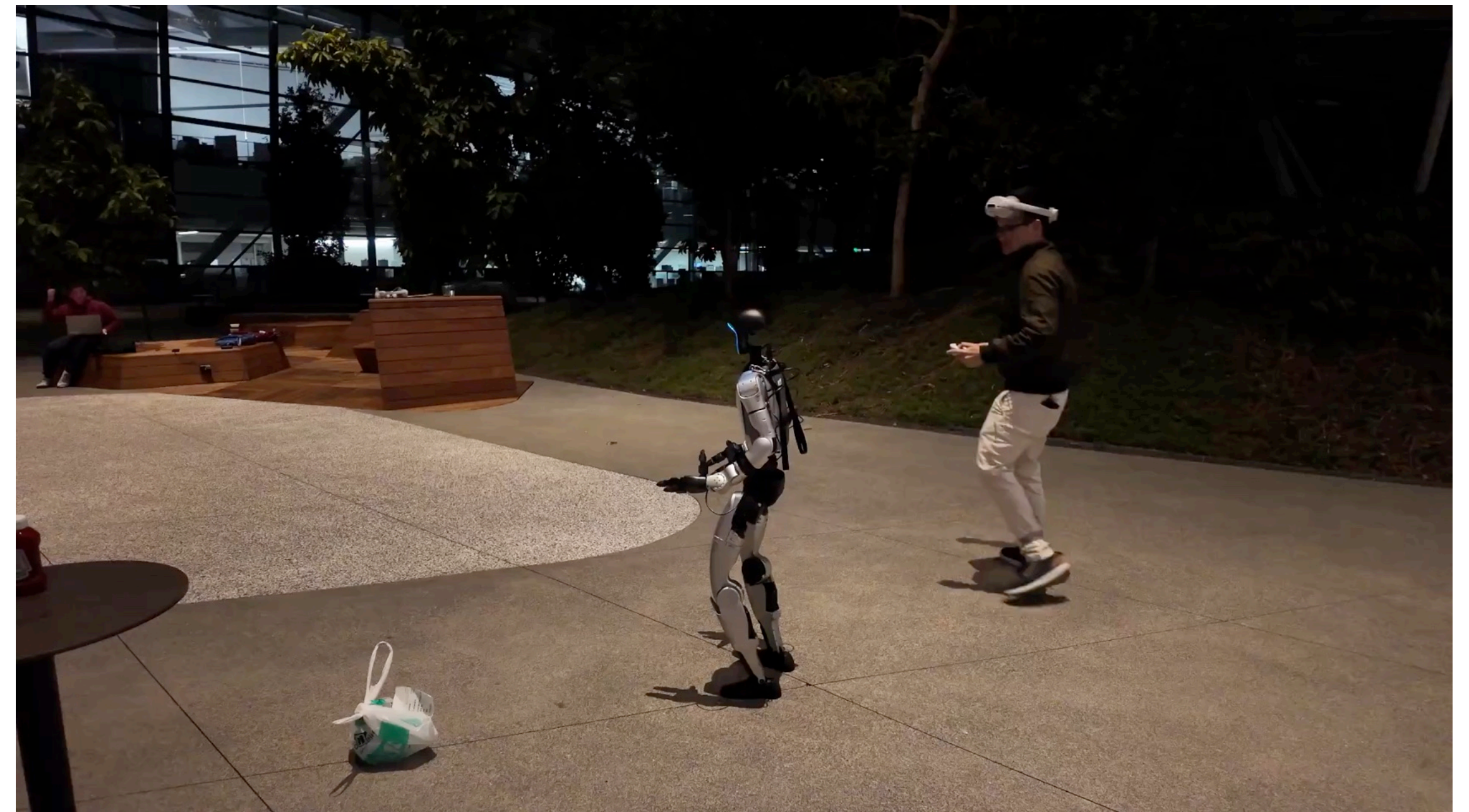
Hand / body tracking for dexterous teleoperation: General recipe

- Use a headset / camera that reports hand and finger poses (usually represented as keypoints / skeletons)
- ‘Retarget’ these poses to the target embodiment (i.e., solve for kinematically feasible robot poses that are closest to the tracked hand/finger targets)
- Compute joint position targets for the target embodiment (e.g., dexterous robot hand)
- Record all intermediate measurements and issued commands
- Same method applies to the entire arm or the whole body



Whole-body teleoperation: challenges

- The robot base is no-longer mounted to a rigid link/bench
- Joint targets cannot be reliably tracked (they're now coupled to lower-body stabilization controllers)
- Kinematically valid poses can be dynamically infeasible



Popular approaches for hand/finger pose tracking

Input	Pros	Cons/Issues
Headset / cameras	Lightweight, least-invasive	Fingers are occluded when grasping objects
Instrumented glove/wearable (e.g., UMI)	Can potentially record occluded poses	Fit, calibration, drift, operator variation
External motion trackers	Versatile, precise	Need external infrastructure

Open-problems / frontiers in data for robot learning

Open challenges / frontiers in data for robot learning

- Force / tactile feedback to teleoperator (bilateral teleop)
- Teleoperation responsiveness (slower than regular human pace)
- Compliance and contact-rich teleoperation
- Demonstrators and policies have observation mismatch
- Role of simulation and data-augmentation is still ripe for exploration
- What kinds of data (expert, recovery, play)?
- How to label / post-process this data?

Questions?